

MeshEcho: Confidence-Aware Route Selection for LoRa Mesh IoT

Zu Gao Zhi Quan
Shenzhen University, Shenzhen, China

Abstract—We compare the current MeshEcho route-selection policy with stylized Meshtastic-like managed flooding and MeshCore-like cached source routing under a shared packet-level LoRa simulation. These are protocol-inspired models, not runs of either project’s firmware. On 20 repeated-pair fading seeds (793 unicasts per policy), MeshEcho’s source-ACK and destination packet-delivery ratios are 0.624/0.633, versus 0.505/0.989 for Meshtastic-like and 0.410/0.448 for MeshCore-like; aggregate transmission airtime is 23.7, 42.4, and 19.5 s per run. Thus MeshEcho obtains more source ACKs than both models in this conditioned workload, but flooding delivers substantially more packets to destinations. On 20 separate, unconditioned random-pair seeds (813 unicasts), Meshtastic-like exceeds MeshEcho in both delivery metrics while using 38.9 versus 152.8 s airtime. A PRR-product route score with matched recovery shows no clear ACK-delivery difference from MeshEcho in either workload. The results identify a workload-dependent protocol trade-off, not firmware-level dominance or a distinct advantage for MeshEcho’s route score.

Index Terms—LoRa mesh, Internet of Things, route selection, fading, packet delivery ratio, airtime

I. INTRODUCTION

LoRa is attractive for low-power IoT because it combines long link budget with inexpensive radios [1]. A star topology is insufficient for obstructed or infrastructure-poor deployments, motivating multi-hop extensions [2]–[4]. Multi-hop forwarding is expensive: a low data rate occupies the channel for a long time, while a flood multiplies collision opportunities. A cached source route reduces redundancy, but one weak hop can invalidate an otherwise useful path.

The practical choice is not only which cached route to rank. Managed flooding can deliver DATA through several paths while spending more transmissions; on-demand source routing spends discovery time and may fail before DATA starts. MeshEcho combines on-demand route discovery with a modeled weakest-hop packet-reception-ratio (PRR) score and a bounded route-miss fallback. We ask how that complete policy compares with two established LoRa mesh design families under identical simulated topology and traffic. We distinguish delivery at the destination from receipt of a reverse ACK at the source; collapsing them would hide an important flooding trade-off.

Our contributions are:

- a same-trace, seed-paired comparison of calibrated MeshEcho with Meshtastic-like and MeshCore-like behavior models on conditioned recurring pairs and an unconditioned random-pair stress case;

- separate ACK, destination-delivery, aggregate TX-airtime, and modeled energy reporting, including an adverse workload where flooding dominates;
- a matched-recovery PRR-product control that tests whether MeshEcho’s max-min score adds value beyond a familiar path-quality metric.

The cross-protocol baselines were added after the earlier MeshEcho holdout had been examined. These comparisons are exploratory, not a new prespecified holdout or an evaluation of production firmware.

II. RELATED WORK

Prior ICC LoRa studies range from analytical and simulation work [5], [6] to testbed-supported evaluation [7], [8]. Multihop surveys and implementations describe flooding, route state, and timing costs [2], [9], [10]. Meshtastic documents managed flooding for broadcasts and, since version 2.6, learned next-hop routing for direct messages [11]. MeshCore documents an initially flooded message whose delivery report can establish a path [12]. Our simplified counterparts isolate flooding and cached source-route behavior; neither reproduces those full implementations. ETX provides the established expected-transmissions reference [13]; we use a matched-recovery PRR-product score to test a closer route-ranking alternative.

III. PROTOCOL AND MODEL

A. Network model

Nodes are uniformly placed in a square and share one half-duplex LoRa channel. Unless noted, packets have 32-byte payloads, 17 dBm transmit power, 125-kHz bandwidth, coding rate 4/5, capture threshold 6 dB, and SF7. Airtime follows the standard LoRa symbol calculation [14]. Log-distance path loss with log-normal shadowing produces SNR; reception is a logistic function of SNR margin, and overlapping packets are accepted only when the desired signal passes capture. The simulator records collisions, route-cache events, and discovery success. Total airtime is the sum of every packet’s TX duration, including control, DATA, ACK, and recovery packets; overlapping transmissions are not de-duplicated into channel-busy time. Energy is calculated from configured TX current and from an always-listening assumption for non-transmitting nodes, not measured battery consumption.

B. Routing policies

The Meshtastic-like model floods each DATA packet with delayed rebroadcast, duplicate suppression, and a reverse ACK along the DATA copy’s realized path. It does not implement Meshtastic 2.6+ direct-message next-hop learning, retry details, or complete firmware semantics. Hence its reverse-path ACK outcome is specific to this harness, especially when DATA reaches a destination by multiple copies.

The MeshCore-like model sends a separate route request (RREQ), waits 2 s for destination candidates, replies along the shortest candidate, and caches the resulting source route for 600 s. It has no route-miss fallback in this comparison. Real MeshCore instead initially floods the message and uses a delivery report to establish a path; our RREQ/RREP abstraction is not a firmware implementation. Both routed models use matched RREQ relay timing when they have a request, but their later transmissions can change interference and candidate exposure.

Current MeshEcho uses the same RREQ collection window and cache lifetime. For each successfully received RREQ hop i , its score maps observed SINR to the simulator’s predicted PRR $\hat{p}_i$. It selects the path maximizing the weakest modeled hop PRR, with shorter paths breaking ties:

$$\hat{p}_i = f_{\text{sim}}(\text{SINR}_i), \quad C_{\text{cal}} = \min_i \hat{p}_i, \quad C_{\text{prod}} = \prod_i \hat{p}_i. \quad (1)$$

The PRR-product comparator uses C_{prod} with MeshEcho’s same TTL-2 route-miss fallback. These scores use model-inferred PRR from received requests, not independently measured link PRR; failed requests are censored. MeshEcho invokes the TTL-2 fallback only if discovery has no candidate. Its default confidence threshold is zero, so a selected route does not launch confidence-triggered duplicate forwarding. Timeout retransmission is disabled for both scored policies. Comparing either with flooding or MeshCore-like measures an entire policy bundle; only the matched-recovery PRR-product row tests the score choice more narrowly.

IV. EXPERIMENTAL DESIGN

A. Two workloads and evidence status

Each run has 50 nodes, 600 s duration, SF7, 4-dB static shadowing, independent reception randomness, and 6-dB temporal block fading every 60 s. There are no application broadcasts. The same topology, application schedule, and per-seed trace hash are used for all four policies. Table II separates the two workload strata; neither is a hardware measurement.

In the recurring-pair case, four directed pairs per seed are chosen from a shared analytical graph with edge PRR ≥ 0.90 and graph distance 2–4 hops. The graph is not supplied to any policy. Every pair has scheduled flows in at least two fading blocks. The case passed a per-seed gate for a complete pair pool, mean graph distance at least two, and at least 10% of direct links below 0.99 static PRR. Its 793 unicasts per policy estimate behavior conditional on connected, repeatedly used pairs; the 51–70 seeds were already inspected in an

earlier MeshEcho-only analysis. Adding the two protocol-family baselines to them is a post-hoc exploratory extension, not a newly untouched holdout.

The separate random-pair case redraws source and destination for each Poisson application flow in an 18-km square; a pair may recur by chance. Its 813 unicasts per policy span seeds 2001–2020. No graph-distance or route-feasibility gate is applied, so some demands can be difficult or disconnected. The parameter choice followed the earlier holdout inspection; these fresh seeds give a robustness check, not a preregistered confirmatory test.

B. Measurements and inference

ACK PDR requires a source-received reverse-path ACK; destination PDR only requires DATA arrival. Each protocol’s seed-level PDR divides its ACK or destination count by that seed’s observed unicasts; the reported mean gives each of the 20 seeds equal weight. For a policy difference d_s , we report $\bar{d} \pm t_{.975,19} s_d / \sqrt{20}$, pairing by seed. Airtime and modeled TX/RX energy are per 600-s run. Flows within a seed are not independent replicates; intervals are nominal and no multiplicity-adjusted superiority claim is made. Routed policies share a 2-s discovery window, 600-s route TTL, and matched request relay timing. Their candidate sets can still diverge after earlier forwarding decisions change interference.

V. RESULTS

Table III reports seed means and Table IV reports paired policy differences. The two scenarios deliberately answer different questions: recurring connected conversations versus unconstrained traffic.

A. Recurring-pair trade-off

On the four connected pairs per seed, MeshEcho exceeds Meshtastic-like by 0.119 in source-ACK PDR while using 18.7 s less aggregate TX airtime and 38.1 J less modeled energy. The destination metric reverses the delivery ranking: Meshtastic-like delivers 784/793 DATA packets, but only 400/793 source ACKs return; MeshEcho delivers 498/793 and returns 490/793 ACKs. The flood’s reverse ACK rule can make this gap large in the simulator and must not be read as a measured Meshtastic failure rate. MeshEcho exceeds MeshCore-like by 0.214 ACK and 0.186 destination PDR at 4.22 s more airtime and 8.63 J more modeled energy. It therefore does not dominate MeshCore-like on cost.

B. Unconditioned random-pair stress

The ranking changes sharply without graph-selected recurring pairs. Meshtastic-like obtains 0.708 ACK and 0.995 destination PDR at 38.9 s aggregate TX airtime; MeshEcho obtains 0.486/0.515 at 152.8 s. Thus the managed-flooding model dominates MeshEcho on both delivery metrics, airtime, and modeled energy in this workload. Its pooled destination count is 809/813, versus MeshEcho’s 415/813. This adverse result is not pooled with the conditioned case. MeshEcho remains ahead of MeshCore-like by 0.105 ACK PDR, but uses

TABLE I
SIMULATOR POLICY MAPPING. THESE ARE IMPLEMENTATION CHOICES IN THIS RESEARCH HARNESS, NOT A SPECIFICATION OF MESHTASTIC OR MESHCORE FIRMWARE.

Policy	First unicast	Later unicast	Source ACK	No route candidate
Meshtastic-like	Managed DATA flood	Managed DATA flood	Reverse realized DATA path	Not applicable
MeshCore-like	Separate RREQ/RREP, then DATA	Cached shortest source route	Reverse selected route	DATA not sent
MeshEcho	Separate RREQ/RREP, then DATA	Cached max-min-PRR route	Reverse selected route	TTL-2 recovery flood
PRR+fb control	Same as MeshEcho; product score	Cached PRR-product route	Reverse selected route	Same TTL-2 fallback

TABLE II

2.1.25 EXPLORATORY FADING WORKLOADS (20 SEEDS AND 4 FLOWS/MIN EACH; 600 S, SF7, 32 B, 17 DBM, 125 KHZ, CR 4/5).

Case	Area / pairs	Seeds / unicasts per policy
Recurring	8.25 km / 4 selected pairs	51-70 / 793
Random	18 km / redraw per flow	2001-2020 / 813

4.40 s more airtime and 8.83 J more modeled energy per run. The random case has 63.4% of static direct links below 0.99 PRR; that link statistic does not establish that every random pair has a viable multi-hop route.

C. Route-score and discovery checks

The matched-recovery PRR-product control attains 0.621 versus MeshEcho’s 0.624 ACK PDR on recurring pairs and 0.486 versus 0.486 on random pairs. Both paired ACK intervals in Table IV include zero, as do their airtime intervals. The max-min score therefore has no demonstrated advantage over this familiar metric under the tested workloads. In the recurring case, MeshEcho and PRR+fb saw identical candidate sets for 81/83 paired discovery keys; in the random case they did so for only 449/804 keys. MeshEcho and MeshCore-like shared only 192/804 random-case candidate sets. Even with matched request timing, subsequent interference and route state differ. These full-run comparisons cannot isolate the score or prove a particular candidate-ranking mechanism caused the ACK gap versus MeshCore-like.

D. Flow outcomes and route-state exposure

Table V counts events across all 20 seeds in each scenario. “Destination only” is a delivered DATA flow without a source-received ACK, not a failed destination delivery. Its large value for the Meshtastic-like model explains why the two PDR metrics tell different stories. Recurring pairs generate hundreds of cache hits for routed policies; random pairs generate only five across all runs. The random workload therefore repeatedly pays discovery costs. MeshEcho records 551 successful discoveries among 807 attempts there, versus 494/807 for MeshCore-like, but this is a complete-protocol difference, not a controlled score comparison.

In the recurring workload, MeshEcho-minus-Meshtastic-like modeled energy is -38.08 J per run (paired 95% CI $[-44.33, -31.83]$), while MeshEcho-minus-MeshCore-like is $+8.63$ J ($[+5.45, +11.81]$). In the random workload, the corresponding differences are $+223.95$ J ($[+206.61, +241.28]$) and $+8.83$ J ($[+7.61, +10.06]$). These values are consequences of

the simulator’s always-listening energy model and must not be interpreted as device battery-life differences.

E. Route-lifetime sensitivity

The earlier 2.1.24 controls use the same current calibrated policy on the recurring-pair seeds. With no temporal fading, MeshEcho reaches 0.717 ACK PDR. Under 6-dB fading, shortening route TTL from 600 to 30 s changes ACK PDR from 0.624 to 0.320 and aggregate TX airtime from 23.7 to 64.8 s. Paired 30-minus-600 s differences are -0.304 ACK PDR ($[-0.393, -0.215]$) and $+41.05$ s airtime ($[+31.40, +50.70]$). This changes the entire subsequent discovery and interference history; it is a policy sensitivity check, not a controlled estimate of one cache-aging mechanism or a reason to prefer longer TTLs under real mobility.

VI. DISCUSSION AND LIMITATIONS

The conditioned recurring case rewards a cached route and its ACK path; the random case imposes repeated route discovery and includes potentially unreachable pairs. These are different estimands, not two measurements of one universal ranking. The random-case reversal is especially important: the simplified flooding model is both more reliable and cheaper in aggregate transmissions there. The design cannot support a blanket claim that source routing always saves airtime or that MeshEcho outperforms Meshtastic and MeshCore products.

For an application that requires a source-confirmed ACK on repeatedly used, statically feasible pairs, MeshEcho’s modeled trade-off is favorable relative to these two simplified baselines. For sporadic random pairs, the flooding model has the stronger result even on aggregate TX airtime. If minimizing transmissions is primary in recurring pairs, the MeshCore-like row is cheaper but loses delivery. These observations are conditional comparisons, not deployment recommendations: changing the traffic mix, channel, firmware routing state, or ACK procedure can change the order.

A. Protocol-model validity

The two named baselines are deliberately stylized. Meshtastic 2.6+ can learn next hops for direct messages; our model never does. MeshCore can flood an initial message and learn from its delivery report; our model requires a separate RREQ/RREP exchange before DATA. Real firmware also has device roles, retry and duty-cycle behavior, routing-state details, and packet formats not represented here. The reverse-path ACK modeled for the Meshtastic-like flood is not a measured property of Meshtastic devices. Consequently,

TABLE III

CURRENT MESH ECHO VERSUS STYLIZED PROTOCOL-FAMILY MODELS AND A MATCHED-RECOVERY SCORE CONTROL. ENTRIES ARE MEANS OF 20 SEED-LEVEL RATIOS OR PER-600-S TOTALS; EACH POLICY HAS THE SAME TRAFFIC TRACE WITHIN A SEED. “PRR+FB” CHANGES ONLY THE ROUTE SCORE TO PRR-PRODUCT WHILE MATCHING MESH ECHO’S TTL-2 ROUTE-MISS FALLBACK. ENERGY IS MODELED, NOT MEASURED.

Workload	Policy	ACK PDR	Dest. PDR	TX airtime (s)	Energy (J)
Recurring (793 flows)	MeshEcho	.624	.633	23.7	47.2
	Meshtastic-like	.505	.989	42.4	85.3
	MeshCore-like	.410	.448	19.5	38.6
	PRR+fb	.621	.633	23.8	47.5
Random (813 flows)	MeshEcho	.486	.515	152.8	303.4
	Meshtastic-like	.708	.995	38.9	79.4
	MeshCore-like	.381	.408	148.4	294.6
	PRR+fb	.486	.514	153.0	303.8

TABLE IV

MESH ECHO MINUS COMPARATOR, PAIRED BY TOPOLOGY SEED ($n = 20$ IN EACH WORKLOAD). ENTRIES ARE MEAN DIFFERENCES [NOMINAL TWO-SIDED 95% t CI]; AIRTIME IS AGGREGATE TX SECONDS PER RUN. EXPLORATORY CONTRASTS ARE NOT MULTIPLICITY-ADJUSTED.

Workload	Comparator	Δ ACK PDR [CI]	Δ dest. PDR [CI]	Δ TX airtime (s) [CI]
Recurring	Meshtastic-like	+ .119 [+ .011, + .227]	− .355 [− .460, − .251]	− 18.70 [− 21.81, − 15.59]
	MeshCore-like	+ .214 [+ .115, + .312]	+ .186 [+ .086, + .286]	+ 4.22 [+ 2.65, + 5.78]
	PRR+fb	+ .003 [− .006, + .011]	+ .000 [− .007, + .007]	− .13 [− .36, + .09]
Random	Meshtastic-like	− .222 [− .273, − .171]	− .481 [− .525, − .436]	+ 113.82 [+ 104.99, + 122.64]
	MeshCore-like	+ .105 [+ .077, + .133]	+ .107 [+ .075, + .140]	+ 4.40 [+ 3.79, + 5.02]
	PRR+fb	− .000 [− .026, + .026]	+ .001 [− .023, + .024]	− .19 [− .55, + .17]

TABLE V

POOLED EVENT COUNTS OVER 20 SEEDS PER WORKLOAD. “DESTINATION ONLY” MEANS DATA DELIVERED WITHOUT A RETURNED SOURCE ACK. DISCOVERY AND CACHE EVENTS ARE NOT DEFINED FOR THE FLOODING MODEL; POOLED COUNTS ARE DIAGNOSTIC AND ARE NOT USED AS INDEPENDENT STATISTICAL REPLICATES.

Workload	Policy	Destination only	Discovery successes/attempts	Route-cache hits
Recurring	MeshEcho	8	59/83	525
	Meshtastic-like	384	—	—
	MeshCore-like	29	55/80	497
	PRR+fb	10	59/83	525
Random	MeshEcho	22	551/807	5
	Meshtastic-like	236	—	—
	MeshCore-like	22	494/807	5
	PRR+fb	21	549/807	5

these numbers compare algorithms in one harness, not actual products. Fairness controls match traffic, PHY, cache TTL for routed policies, and RREQ timing where applicable; they cannot equalize inherently different packet exchanges or subsequent collision histories.

B. Statistical and physical limits

The 51–70 baseline comparisons reuse already-viewed holdout seeds, and the new random-pair scenario was designed after that inspection. Nominal paired CIs describe seed-to-seed variation but do not create confirmatory evidence or correct the several exploratory contrasts. Twenty topology seeds, rather than 793 or 813 flows, are the independent units. The recurring case selects just four qualified pairs per seed and favors cache reuse; the random case has no graph-feasibility gate. The two cases also change area, so their absolute differences do not isolate pair scheduling.

The packet-level radio model includes path loss, shadowing, half-duplex collisions, and capture, but not waveform interference, modem timing, antenna changes, enforced regulatory

duty cycle, or an empirical channel trace. Modeled PRR is inferred from successfully received requests and does not account for censored requests or measured packet losses. Aggregate TX airtime is not channel-busy time, and the always-listening energy model is not a battery measurement. A hardware or hardware-in-the-loop study with actual firmware, packet traces, and measured energy is needed before deployment claims. A separate, prospectively frozen scenario and a fixed-candidate score test would be needed for stronger causal claims.

VII. REPRODUCIBILITY

Version 2.1.25 provides source, tests, per-seed CSVs, a recurring-pair summary CSV, and reproduction commands for both four-policy experiments. The raw CSVs contain application denominators and trace hashes plus recorded discovery candidates for routed policies. The 51–70 calibrated and PRR+fb rows reproduce the 2.1.24 holdout exactly; the additional baselines and seeds 2001–2020 are new exploratory evidence. Numerical claims were independently recomputed

from the raw counts. No human participants or personal data are involved. AI-assisted drafting and code review were used.

VIII. CONCLUSION

In recurring connected conversations, MeshEcho's source-ACK delivery exceeds both stylized baselines, but Meshtastic-like reaches far more destinations and MeshCore-like uses less aggregate TX airtime. Under unconditioned random demands, Meshtastic-like dominates MeshEcho in both delivery metrics and cost. MeshEcho and matched-recovery PRR-product are nearly identical in both cases. These results delimit where the current simulated policy helps; they do not establish score novelty, firmware superiority, or physical-network performance.

REFERENCES

- [1] A. Augustin, J. Yi, T. Clausen, and W. M. Townsley, "A study of LoRa: Long range & low power networks for the internet of things," *Sensors*, vol. 16, no. 9, p. 1466, 2016.
- [2] A. W.-L. Wong, S. L. Goh, M. K. Hasan, and S. Fattah, "Multihop and mesh for LoRa networks: Recent advancements, issues, and recommended applications," *ACM Computing Surveys*, vol. 56, no. 6, pp. 1–43, 2024.
- [3] J. R. Cotrim and J. H. Kleinschmidt, "LoRaWAN mesh networks: A review and classification of multihop communication," *Sensors*, vol. 20, no. 15, p. 4273, 2020.
- [4] R. P. Centelles, F. Freitag, R. Meseguer, and L. Navarro, "Beyond the star of stars: An introduction to multihop and mesh for LoRa and LoRaWAN," *IEEE Pervasive Computing*, vol. 20, no. 2, pp. 63–72, 2021.
- [5] L.-T. Tu, A. Bradai, and Y. Pousset, "A new closed-form expression of the coverage probability for different QoS in LoRa networks," in *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*, 2020, pp. 1–6.
- [6] O. Afisiadis, A. Burg, and A. Balatsoukas-Stimming, "Coded LoRa frame error rate analysis," in *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*, 2020, pp. 1–6.
- [7] T.-H. To and A. Duda, "Simulation of LoRa in NS-3: Improving LoRa performance with CSMA," in *2018 IEEE International Conference on Communications (ICC)*, 2018, pp. 1–7.
- [8] E. M. Rochester, A. M. Yousuf, B. Ousat, and M. Ghaderi, "Lightweight carrier sensing in LoRa: Implementation and performance evaluation," in *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*, 2020, pp. 1–6.
- [9] J. M. Sole, R. Pueyo Centelles, F. Freitag, and R. Meseguer, "Implementation of a LoRa mesh library," *IEEE Access*, vol. 10, pp. 113 158–113 171, 2022.
- [10] R. Pueyo Centelles, R. Meseguer, F. Freitag, R. Baig Viñas, and L. Navarro, "A minimalistic distance-vector routing protocol for LoRa mesh networks," *IEEE Access*, vol. 12, pp. 128 941–128 962, 2024.
- [11] Meshtastic Project, "Mesh algorithm: Managed flooding and next-hop routing," <https://meshtastic.org/docs/overview/mesh-algo/>, 2026, accessed 2026-10-03.
- [12] MeshCore Project, "MeshCore frequently asked questions," <https://github.com/meshcore-dev/MeshCore/blob/main/docs/faq.md>, 2026, accessed 2026-10-03.
- [13] D. S. J. De Couto, D. Aguayo, J. Bicket, and R. Morris, "A high-throughput path metric for multi-hop wireless routing," in *Proceedings of the 9th Annual International Conference on Mobile Computing and Networking*, ser. MobiCom '03, 2003, pp. 134–146.
- [14] Semtech Corporation, *AN1200.13: LoRa Modem Designer's Guide*, 2013, application note.